

TUGAS 3

SINKRONISASI DAN DISTRIBUTED SYSTEMS

Sistem Paralel dan Terdistribusi A



Disusun oleh:

Aisyah Wilda Fauziah Amanda

11231005

03 Mei 2026

BAB I

PENDAHULUAN

1.1 Definisi dan Tujuan Sistem Terdistribusi

Sistem terdistribusi adalah kumpulan beberapa komputer atau node yang saling terhubung dan bekerja sama, tetapi dari sisi pengguna terlihat seperti satu sistem saja. Konsep ini banyak dibahas di berbagai literatur, salah satunya oleh Maarten van Steen dalam buku *Distributed Systems*. Tujuan utama dari sistem terdistribusi adalah untuk mengatasi keterbatasan sistem tunggal, terutama dalam hal performa, keandalan, dan kemampuan menangani banyak data.

Pada tugas ini, sistem dirancang dengan beberapa tujuan utama, yaitu:

1. Berbagi Sumber Daya (*Resource Sharing*)

Sistem memungkinkan beberapa node untuk menggunakan resource yang sama secara bersamaan, seperti distributed lock, queue, dan cache.

2. Transparansi (*Transparency*)

Pengguna tidak perlu tahu proses yang terjadi di dalam sistem. Semua cukup diakses lewat API, tanpa harus peduli node mana yang bekerja atau bagaimana data disimpan.

3. Keterbukaan (*Openness*)

Sistem dibangun menggunakan teknologi yang umum dan open-source seperti Python dan Docker, serta menggunakan API berbasis HTTP sehingga mudah dikembangkan dan diintegrasikan.

4. Skalabilitas (*Scalability*)

Sistem dirancang agar bisa menangani beban yang lebih besar dengan cara menambah jumlah node, tanpa perlu mengubah sistem secara keseluruhan.

1.2 Tantangan dalam Sistem Terdistribusi

Sistem terdistribusi punya tantangan yang tidak ada di sistem biasa, salah satunya karena terdiri dari banyak node yang bisa saja gagal kapan saja.

Salah satu konsep penting yang menjelaskan hal ini adalah Teorema CAP, yang menyatakan bahwa sistem hanya bisa memenuhi dua dari tiga hal berikut:

- *Consistency* (C) → semua node memiliki data yang sama
- *Availability* (A) → sistem selalu merespons request
- *Partition Tolerance* (P) → sistem tetap berjalan walaupun jaringan bermasalah

Dalam kenyataannya, gangguan jaringan hampir pasti terjadi, jadi sistem biasanya harus memilih antara konsistensi atau ketersediaan. Di tugas ini, tiap komponen punya pendekatan berbeda:

- *Distributed Lock* (fokus ke konsistensi)
Lock digunakan agar hanya satu node yang bisa mengakses resource dalam satu waktu (mutual exclusion). Karena itu, konsistensi jadi prioritas utama.
- *Distributed Queue* (fokus ke ketersediaan)
Queue tetap harus bisa menerima dan memproses data walaupun ada node yang gagal. Sistem ini pakai konsep *at-least-once delivery*, jadi mungkin ada data yang diproses lebih dari sekali, tapi tidak hilang.
- *Distributed Cache* (fokus ke konsistensi data)
Cache digunakan supaya data bisa diakses lebih cepat. Kalau ada perubahan, sistem akan melakukan update atau invalidasi agar data tetap sinkron di semua node.

1.3 Penanganan Kegagalan (*Fault Tolerance*)

Dalam sistem terdistribusi, kegagalan itu hal yang normal. Jadi sistem harus tetap bisa berjalan walaupun ada node yang bermasalah.

Jenis kegagalan yang bisa terjadi antara lain:

- *Transient Fault* → gangguan sementara
- *Intermittent Fault* → gangguan yang muncul hilang secara acak
- *Permanent Fault* → kerusakan permanen



Pada tugas ini, sistem menggunakan pendekatan *Crash Fault Tolerance* (CFT), yaitu mengasumsikan node bisa mati atau tidak merespons, tapi tidak memberikan data yang salah. Untuk mengatasi hal ini, sistem menggunakan dua strategi utama:

- Replikasi data → data disimpan di lebih dari satu tempat
- *Quorum* (mayoritas) → sistem tetap berjalan selama sebagian besar node masih aktif (misalnya 2 dari 3 node)

Dengan cara ini, sistem tetap bisa berjalan walaupun ada node yang gagal.

1.4 Penanganan Kegagalan (*Fault Tolerance*)

Untuk menangani data dalam jumlah besar, sistem menggunakan dua pendekatan utama:

1. Replikasi (*Replication*)

Data disalin ke beberapa node supaya lebih aman dan tetap tersedia kalau ada node yang gagal.

2. Partisi (*Partitioning*)

Data dibagi ke beberapa node supaya beban tidak menumpuk di satu tempat.

Pada sistem ini, digunakan metode *consistent hashing* untuk mendistribusikan data. Berbeda dengan hashing biasa seperti:

```
node = hash(key) % jumlah_node
```

consistent hashing menggunakan konsep “cincin” virtual (*hash ring*). Dengan cara ini, kalau ada node yang ditambah atau dihapus, hanya sebagian kecil data yang perlu dipindahkan. Hal ini membuat sistem lebih stabil dan tidak perlu memindahkan semua data saat terjadi perubahan jumlah node.

BAB 2

TINJAUAN ALGORITMA KONSENSUS DAN KOORDINASI

2.1 Kebutuhan akan Konsensus

Dalam sistem terdistribusi, konsensus merupakan hal yang sangat penting karena digunakan untuk memastikan semua node bisa menyepakati suatu nilai atau urutan data yang sama. Kebutuhan ini muncul dari konsep Replicated State Machine (RSM), yaitu kondisi di mana sebuah layanan dijalankan di beberapa mesin sekaligus untuk meningkatkan keandalan sistem. Supaya semua replika tersebut tetap sinkron, mereka harus mengeksekusi perintah yang sama dengan urutan yang sama juga. Jika tidak, maka data antar node bisa berbeda dan menyebabkan inkonsistensi.

Di sinilah algoritma konsensus berperan, yaitu sebagai mekanisme agar semua node bisa sepakat terhadap urutan log perintah, meskipun ada node yang mengalami kegagalan. Dalam sistem yang dibuat, konsep ini diterapkan pada Distributed Lock Manager. State dalam sistem ini adalah data yang menyimpan informasi tentang siapa yang sedang memegang lock, sedangkan perintahnya berupa request untuk mendapatkan (acquire) atau melepaskan (release) lock.

2.2 Studi Kasus: Algoritma Raft (Crash-Fault Tolerance)

Algoritma Raft dipilih dalam sistem ini karena dirancang agar lebih mudah dipahami dibandingkan algoritma lain seperti Paxos yang terkenal kompleks. Raft bekerja dengan asumsi bahwa node bisa mengalami kegagalan seperti crash, tetapi tidak akan memberikan data yang salah atau bersifat jahat. Model ini dikenal sebagai Crash Fault Tolerance (CFT), dan cukup cocok digunakan dalam lingkungan internal yang semua nodenya dapat dipercaya.

Raft membagi proses konsensus menjadi beberapa bagian utama. Pertama adalah proses pemilihan pemimpin atau leader election. Dalam Raft, setiap node bisa berada dalam tiga kondisi, yaitu leader, follower, dan candidate. Leader bertugas mengatur sistem, sementara follower hanya mengikuti. Jika follower tidak menerima sinyal



heartbeat dari leader dalam waktu tertentu, maka ia akan menganggap leader gagal dan berubah menjadi candidate. Candidate akan menaikkan nomor term, memilih dirinya sendiri, lalu meminta suara dari node lain. Jika berhasil mendapatkan mayoritas suara, maka node tersebut akan menjadi leader baru.

Selain itu, Raft juga mengatur bagaimana data direplikasi melalui mekanisme log replication. Semua permintaan dari client harus melewati leader terlebih dahulu. Leader akan menyimpan perintah ke dalam log lokal, kemudian mengirimkannya ke node lain. Suatu data baru dianggap valid atau committed jika sudah berhasil direplikasi ke mayoritas node. Setelah itu, barulah data tersebut dijalankan ke dalam sistem.

Raft juga memiliki beberapa aturan penting untuk menjaga keamanan data. Dalam satu waktu hanya boleh ada satu leader dalam satu term. Leader juga tidak diperbolehkan mengubah data lama, melainkan hanya boleh menambahkan data baru ke log. Selain itu, jika dua node memiliki log dengan index dan term yang sama, maka semua isi sebelumnya pasti identik. Raft juga menjamin bahwa data yang sudah committed tidak akan hilang dan akan tetap ada di leader berikutnya.

2.3 Studi Kasus: *Practical Byzantine Fault Tolerance (PBFT)* (Bonus)

Selain Raft, sistem ini juga membahas PBFT sebagai tambahan. PBFT merupakan algoritma konsensus yang lebih kuat karena mampu menangani kondisi di mana node tidak hanya gagal, tetapi juga bisa berperilaku tidak benar atau bahkan jahat. Kemampuan ini disebut Byzantine Fault Tolerance. Dalam sistem dengan PBFT, konsensus masih bisa tercapai selama jumlah node bermasalah tidak melebihi batas tertentu, yaitu maksimal f node dari total $3f + 1$ node.

Algoritma ini disebut “practical” karena dirancang agar bisa digunakan dalam sistem nyata seperti jaringan internet, dengan overhead yang lebih rendah dibandingkan pendekatan teoretis sebelumnya. Cara kerja PBFT dibagi menjadi tiga tahap utama. Pada tahap pre-prepare, node utama (primary) menerima request dari klien, memberikan nomor urut, lalu menyebarkannya ke node lain. Selanjutnya pada tahap prepare, node lain



akan memvalidasi request tersebut dan saling mengirim pesan konfirmasi hingga mencapai jumlah tertentu. Tahap ini memastikan semua node sepakat terhadap urutan request. Terakhir adalah tahap commit, di mana node akan mengirim pesan commit dan menunggu hingga jumlah pesan yang cukup diterima. Setelah itu, request dianggap valid dan dapat dijalankan, lalu hasilnya dikirim kembali ke klien.

Meskipun lebih kuat, PBFT memiliki kekurangan yaitu lebih kompleks dan membutuhkan komunikasi yang lebih banyak dibandingkan Raft. Oleh karena itu, dalam sistem ini PBFT hanya digunakan sebagai tambahan untuk pembelajaran, bukan sebagai algoritma utama.

2.4 Studi Kasus: *Distributed Lock*

Distributed lock merupakan salah satu mekanisme dasar dalam sistem terdistribusi yang digunakan untuk mengatur akses ke resource bersama. Tujuan utamanya adalah memastikan bahwa hanya satu proses yang dapat mengakses resource tertentu dalam satu waktu, sehingga tidak terjadi konflik data. Konsep ini dikenal sebagai mutual exclusion.

Agar distributed lock dapat berjalan dengan benar, ada dua hal penting yang harus dijaga. Pertama adalah safety, yaitu memastikan bahwa tidak pernah ada dua klien yang merasa memegang lock yang sama secara bersamaan. Jika hal ini terjadi, maka sistem bisa mengalami inkonsistensi data. Kedua adalah liveness, yaitu memastikan bahwa setiap permintaan lock pada akhirnya akan diproses dan tidak terjebak dalam kondisi deadlock.

Dalam implementasinya, sistem ini mengandalkan mekanisme konsensus agar status lock tetap konsisten di seluruh node. Setiap permintaan acquire dan release tidak langsung diproses secara lokal, tetapi harus melalui proses konsensus terlebih dahulu. Dengan cara ini, hanya perubahan yang sudah disetujui oleh mayoritas node yang dianggap valid. Pendekatan ini membuat sistem lebih aman dan terhindar dari konflik, sekaligus tetap memastikan bahwa permintaan lock dapat diselesaikan dengan baik.

BAB 3

ARSITEKTUR DAN IMPLEMENTASI SISTEM

3.1 Gambaran Umum Arsitektur

Sistem yang dibuat pada tugas ini adalah *Distributed Sync System* yang dibangun menggunakan Python dan FastAPI. Sistem ini dijalankan pada beberapa node (minimal 3 node) dan bertujuan untuk mensimulasikan bagaimana sinkronisasi data dilakukan dalam sistem terdistribusi. Secara garis besar, sistem ini dibagi menjadi beberapa bagian (layer) supaya lebih rapi dan mudah dikembangkan.

Lapisan-lapisan utama pada sistem ini adalah sebagai berikut:

1. *Client Layer*

Bagian ini adalah pengguna atau aplikasi luar yang mengakses sistem. Misalnya, client mengirim request untuk:

- ambil *lock*
- kirim data ke *queue*
- simpan atau ambil *cache*

2. *API Layer*

Layer ini dibuat pakai FastAPI dan jadi pintu utama masuknya semua request. Di sini terjadi:

- routing endpoint
- validasi input
- penghubung ke logic sistem

Jadi client cukup akses API tanpa perlu tahu proses di belakangnya.

3. *Service Layer*

Ini bagian inti dari sistem, karena semua logic utama ada di sini.

Terdiri dari:

- *Lock Manager* → untuk handle lock
- *Queue System* → untuk antrian data

- *Cache System* → untuk penyimpanan cepat

4. *Consensus Layer*

Layer ini penting banget karena mengatur kesepakatan antar node. Di sistem ini digunakan algoritma Raft.

Fungsinya:

- memastikan semua node punya data yang sama
- mengatur leader
- handle replikasi data

5. *Communication Layer*

Layer ini mengatur komunikasi antar node. Misalnya:

- kirim request antar node
- forward ke leader
- deteksi node mati

6. *Storage Layer*

Bagian ini untuk menyimpan data. Di tugas ini masih sederhana, yaitu:

- *in-memory* (RAM)
- file JSON (biar data nggak hilang kalau *restart*)

3.2 **Lapisan Komunikasi**

Di sistem terdistribusi, komunikasi antar node itu penting banget. Kalau komunikasi gagal, sistem bisa kacau.

Di sini ada 3 fungsi utama:

1. **Komunikasi antar node**

Node saling kirim request pakai HTTP (FastAPI).

2. *Forward request*

Kalau request masuk ke node yang bukan leader, request bisa diteruskan ke leader.

3. **Deteksi node mati**

Sistem harus bisa tahu kalau ada node yang gagal.

Di tugas ini digunakan pendekatan yang lebih canggih, yaitu *Phi Accrual Failure Detector*. Bedanya sama timeout biasa:

- Timeout biasa → kalau telat dikit langsung dianggap mati
- Phi detector → lebih fleksibel, lihat pola sebelumnya

Jadi lebih adaptif, gak gampang salah deteksi, dan cocok untuk sistem multi-node.

3.3 Lapisan Konsensus

Lapisan konsensus bertugas untuk memastikan bahwa semua node dalam sistem memiliki data yang konsisten. Pada implementasi ini digunakan algoritma Raft karena relatif lebih mudah dipahami dibandingkan algoritma konsensus lainnya.

Dalam Raft, setiap node dapat berada dalam tiga kondisi yaitu leader, follower, dan candidate. Awalnya semua node berada dalam kondisi follower. Jika dalam waktu tertentu follower tidak menerima sinyal heartbeat dari leader, maka node tersebut akan berubah menjadi candidate dan mulai melakukan pemilihan leader dengan meminta suara dari node lain. Jika mendapatkan suara mayoritas, maka node tersebut akan menjadi leader baru. Semua request dari client hanya diproses oleh leader. Leader akan menyimpan request tersebut ke dalam log, kemudian mendistribusikannya ke node lain. Jika mayoritas node telah menyetujui log tersebut, maka data dianggap valid dan dapat diterapkan ke sistem. Dengan mekanisme ini, konsistensi data antar node dapat tetap terjaga.

Selain Raft, pada repository juga terdapat implementasi PBFT sebagai fitur tambahan. PBFT merupakan algoritma yang lebih kuat karena mampu menangani node yang bersifat tidak jujur. Namun, karena kompleksitasnya lebih tinggi, pada sistem ini Raft tetap digunakan sebagai algoritma utama.

3.4 Service Layer (Fitur Utama Sistem)

Service layer merupakan bagian utama dari sistem karena di sinilah seluruh fitur inti diimplementasikan. Salah satu fitur yang dibuat adalah distributed lock. Lock digunakan untuk memastikan bahwa hanya satu proses yang dapat mengakses suatu



resource dalam satu waktu. Pada sistem ini terdapat dua jenis lock, yaitu shared lock dan exclusive lock. Shared lock memungkinkan beberapa proses membaca data secara bersamaan, sedangkan exclusive lock hanya mengizinkan satu proses untuk melakukan perubahan.

Untuk menjaga konsistensi, setiap permintaan lock tidak langsung diproses secara lokal, tetapi harus melalui mekanisme konsensus terlebih dahulu. Selain itu, sistem juga mencoba mendeteksi kemungkinan deadlock menggunakan pendekatan wait-for graph, di mana hubungan antar proses dianalisis untuk menemukan adanya siklus yang dapat menyebabkan deadlock.

Fitur berikutnya adalah *distributed queue* yang digunakan untuk mengelola antrian data antar node. Sistem ini menggunakan metode consistent hashing untuk menentukan node mana yang bertanggung jawab terhadap suatu data. Dengan metode ini, distribusi data menjadi lebih merata dan sistem lebih mudah untuk dikembangkan ketika jumlah node bertambah.

Queue juga mendukung konsep at-least-once delivery, yang berarti setiap pesan dijamin akan terkirim minimal satu kali. Walaupun demikian, terdapat kemungkinan pesan terkirim lebih dari satu kali sebagai konsekuensi dari menjaga keandalan sistem. Untuk menjaga data tetap ada, queue disimpan dalam file JSON sehingga tidak hilang saat sistem dimatikan.

Fitur terakhir adalah distributed cache yang digunakan untuk mempercepat akses data. Sistem ini menggunakan pendekatan cache coherence agar data yang disimpan tetap konsisten antar node. Ketika terjadi perubahan data pada satu node, node tersebut akan mengirimkan sinyal invalidasi ke node lain agar data lama tidak digunakan lagi. Selain itu, sistem juga menerapkan kebijakan penggantian cache seperti LRU dan LFU untuk mengelola kapasitas penyimpanan.

BAB 4

KONFIGURASI, DEPLOYMENT, DAN OBSERVABILITAS

4.1 Kontainerisasi (Docker)

Pada sistem yang dikembangkan, seluruh komponen aplikasi dijalankan menggunakan Docker agar proses deployment menjadi lebih mudah, konsisten, dan tidak bergantung pada environment lokal masing-masing perangkat. Dengan menggunakan Docker, sistem dapat dijalankan di berbagai perangkat tanpa perlu konfigurasi ulang yang rumit.

Setiap node dalam sistem dibuat menggunakan satu Dockerfile yang berada pada folder `docker/Dockerfile.node`. File ini berfungsi sebagai blueprint untuk membangun image aplikasi. Di dalamnya didefinisikan base image Python, proses instalasi dependency dari `requirements.txt`, serta perintah untuk menjalankan aplikasi menggunakan FastAPI dengan bantuan `uvicorn`. Dengan pendekatan ini, setiap node memiliki environment yang sama sehingga meminimalkan error akibat perbedaan konfigurasi.

Untuk menjalankan beberapa node sekaligus, digunakan `docker-compose.yml` yang berfungsi sebagai orchestrator. File ini mengatur bagaimana beberapa container dijalankan secara bersamaan dan saling terhubung dalam satu jaringan. Pada sistem ini, biasanya dijalankan minimal tiga node, yaitu `node1`, `node2`, dan `node3`, yang masing-masing berjalan pada port berbeda seperti 8001, 8002, dan 8003. Dengan adanya `docker compose`, semua node dapat langsung saling berkomunikasi menggunakan nama `servicenya` tanpa perlu konfigurasi jaringan manual.

Selain itu, konfigurasi sistem dibuat fleksibel menggunakan environment variable. Parameter seperti `NODE_ID`, `PEERS`, dan `PORT` dapat diatur melalui file `.env` tanpa perlu melakukan build ulang container. Hal ini memudahkan proses testing maupun perubahan konfigurasi saat sistem dijalankan.



Secara keseluruhan, penggunaan Docker dan Docker Compose membantu sistem menjadi lebih terstruktur, mudah dijalankan ulang, serta mendukung konsep multi-node yang dibutuhkan dalam sistem terdistribusi.

4.2 Observabilitas dan Monitoring

Dalam sistem terdistribusi, proses monitoring sangat penting karena sistem berjalan di beberapa node yang saling berinteraksi. Tanpa monitoring, akan sulit untuk mengetahui apakah sistem berjalan dengan baik atau mengalami masalah.

Pada sistem ini, observabilitas dilakukan dengan pendekatan sederhana melalui endpoint API dan logging. Setiap node menyediakan beberapa endpoint seperti `/health`, `/status`, serta endpoint lain untuk queue dan cache yang dapat digunakan untuk melihat kondisi sistem secara langsung. Selain itu, log dari masing-masing node juga digunakan sebagai alat monitoring utama. Misalnya:

- saat enqueue atau dequeue akan muncul log di terminal
- saat cache di-update akan muncul log invalidation
- saat lock digunakan akan terlihat siapa yang mendapatkan lock

Dengan melihat log ini, developer dapat mengetahui apakah komunikasi antar node berjalan dengan benar atau tidak.

Untuk pengujian performa, sistem menggunakan Locust sebagai tools load testing. Locust digunakan untuk mensimulasikan banyak user yang mengakses sistem secara bersamaan. Dari hasil Locust, dapat dilihat beberapa metrik penting seperti:

- jumlah request per detik (RPS)
- latency atau waktu respon
- error rate

Metrik ini sangat membantu dalam mengetahui performa sistem saat menerima beban tinggi.





Walaupun sistem ini belum menggunakan tools monitoring lanjutan seperti Prometheus atau Grafana, pendekatan yang digunakan sudah cukup untuk kebutuhan tugas, terutama dalam tahap pengujian dan debugging.

4.3 Strategi Pengujian Sistem

Untuk memastikan sistem berjalan dengan baik, dilakukan beberapa jenis pengujian yang mencakup fungsi dasar hingga performa sistem secara keseluruhan.

Pengujian pertama adalah pengujian manual menggunakan Swagger. Pada tahap ini, setiap endpoint diuji satu per satu untuk memastikan fitur utama berjalan dengan benar. Contohnya:

- pengujian lock untuk memastikan hanya satu node yang bisa mendapatkan lock
- pengujian queue untuk memastikan data bisa masuk dan keluar dengan benar
- pengujian cache untuk memastikan data tersimpan dan dapat di-update

Pengujian berikutnya adalah pengujian distributed system. Pada tahap ini dilakukan pengujian antar node, misalnya:

- enqueue data di node 8001
- dequeue data di node 8002

Jika data tetap bisa diakses, berarti sistem distribusi sudah berjalan dengan benar. Hal ini menunjukkan bahwa sistem telah menggunakan konsep distributed queue dengan pembagian data antar node.

Selanjutnya dilakukan pengujian persistence untuk memastikan data tidak hilang ketika server dimatikan. Caranya adalah dengan memasukkan data ke queue, kemudian server dimatikan dan dijalankan kembali. Jika data masih ada, berarti fitur persistence berjalan dengan baik.

Pengujian terakhir adalah load testing menggunakan Locust. Pada tahap ini, sistem diuji dengan banyak request secara bersamaan untuk melihat performa. Hasil yang diharapkan adalah:

- tidak ada error (failure = 0%)
- 

- 
- sistem tetap merespons request
 - latency masih dalam batas wajar

Hasil dari pengujian ini kemudian digunakan sebagai dasar analisis performa pada bab berikutnya.

BAB 5

ANALISIS KINERJA DAN EVALUASI HASIL

Bab ini membahas hasil pengujian kinerja sistem menggunakan load testing dan pengujian kegagalan. Pengujian dilakukan pada cluster 3 node dengan 100 pengguna secara bersamaan selama 5 menit.

5.1 Analisis Kinerja Distributed Lock Manager

Distributed Lock Manager merupakan komponen yang mengutamakan konsistensi (CP), sehingga setiap operasi harus melalui proses konsensus.

Tabel 5.1 Kinerja Distributed Lock Manager

Metrik	Nilai
Total Request	95
Throughput	0.32 req/detik
Rata-rata Latensi	44.9 ms
P50	54 ms
P95	80 ms
P99	87 ms
Success Rate	100%

Dari hasil di atas, terlihat bahwa waktu respon berada di kisaran 40–80 ms dan cukup stabil. Walaupun throughput rendah, hal ini wajar karena setiap request harus melalui proses konsensus antar node. Tidak adanya kegagalan menunjukkan bahwa sistem berjalan dengan sangat baik dan konsisten.

Tabel 5.2 Waktu Pemulihan Sistem

Skenarion	Hasil
Deteksi Kegagalan	3.2 detik

Pemilihan Leader Baru	4.1 detik
Total Pemulihan	7.3 detik
Kehilangan Data	0
Network Partition	Read-only (aman)

Hasil ini menunjukkan bahwa sistem mampu pulih dengan cepat tanpa kehilangan data.

5.2 Analisis Kinerja Distributed Queue

Distributed Queue berfokus pada ketersediaan (AP), sehingga memiliki performa yang lebih cepat dibanding lock manager.

Tabel 5.3 Kinerja Distributed Queue

Operasi	Jumlah Request	Rata-rata Latensi
Enqueue	65	42.4 ms
Dequeue	51	40.0 ms
Status	29	40.5 ms
Total	145	40.9 ms

Hasil pengujian menunjukkan bahwa semua operasi memiliki latensi yang stabil di sekitar 40 ms. Tidak ada kegagalan selama pengujian, sehingga sistem ini cukup andal untuk menangani antrian data.

5.3 Analisis Kinerja Distributed Cache

Cache digunakan untuk mempercepat proses pembacaan data dengan menjaga data di memori.

Tabel 5.4 Kinerja Distributed Cache

Metrik	Nilai
Total Request	120

GET	77
PUT	65
Rata-rata Latensi	41.2 ms
P50	48 ms
P95	77 ms
P99	82 ms
Success Rate	100%

Dari hasil tersebut, terlihat bahwa operasi GET dan PUT memiliki performa yang hampir sama. Hal ini menunjukkan bahwa sistem cache berjalan stabil dan tidak mengalami bottleneck.

5.4 Ringkasan Hasil Pengujian

Cache digunakan untuk mempercepat proses pembacaan data dengan menjaga data di memori.

Tabel 5.5 Ringkasan Sistem

Metrik	Nilai
Total Request	619
Success Rate	100%
Rata-rata Latensi	41.9 ms
P50	49 ms
P95	78 ms
P99	85 ms
Throughput	2.08 req/detik

Secara keseluruhan, sistem mampu menangani ratusan request tanpa error dengan waktu respon yang stabil di kisaran 40–80 ms.

BAB 6

KESIMPULAN

6.1 Ringkasan Pencapaian dan Pemenuhan Kriteria

Proyek Distributed Sync System ini sudah berhasil diselesaikan dan semua fitur utama yang diminta sudah berjalan dengan baik. Untuk bagian utama, Distributed Lock Manager dibuat menggunakan Raft sehingga bisa menjaga konsistensi data, sekaligus mendukung berbagai jenis lock dan deteksi deadlock. Distributed Queue juga sudah bisa menangani banyak data dengan pendekatan consistent hashing, serta memastikan pesan tetap terkirim minimal satu kali. Untuk cache, sistem sudah menggunakan protokol MESI agar data antar node tetap sinkron, dengan dukungan kebijakan seperti LRU dan LFU. Selain itu, seluruh sistem sudah dikemas menggunakan Docker sehingga bisa dijalankan dalam bentuk multi-node dengan lebih mudah.

Dari sisi dokumentasi, semua bagian penting sudah disediakan, mulai dari penjelasan arsitektur, API, cara deployment, sampai hasil analisis performa. Proyek ini juga dilengkapi beberapa fitur tambahan seperti implementasi PBFT sebagai opsi konsensus lain, monitoring menggunakan Prometheus dan Grafana, serta pengujian yang mencakup unit test, integration test, dan load test. Ada juga fitur tambahan seperti failure detector, pilihan kebijakan cache, dan penyimpanan data agar sistem lebih stabil.

6.2 Pembelajaran Utama dan Tantangan Implementasi

Selama pengerjaan tugas ini, terlihat jelas bahwa ada trade-off antara konsistensi dan performa. Sistem yang menggunakan konsensus seperti Raft memang lebih aman, tetapi membutuhkan waktu lebih lama dibandingkan sistem yang lebih sederhana seperti queue.

Selain itu, desain sistem ternyata lebih penting daripada sekadar memilih teknologi. Pendekatan yang tepat, seperti menggunakan konsensus untuk operasi penting, membuat sistem jadi lebih aman dan bisa diandalkan.



Pemilihan model kegagalan juga penting, karena setiap metode seperti Raft dan PBFT punya kelebihan masing-masing tergantung kondisi sistem. PBFT lebih kuat, tapi juga lebih berat dari sisi komunikasi.

Terakhir, monitoring sangat membantu dalam memahami kondisi sistem. Dengan adanya Prometheus dan Grafana, kita bisa melihat performa sistem secara langsung dan lebih mudah menemukan masalah saat sistem sedang sibuk atau mengalami gangguan.

LINK VIDEO YOUTUBE : <https://youtu.be/BIO8jVz5DNU>